

COMP90038 Algorithms and Complexity

Computational Models and Asymptotic Notation

Junhao Gan

Week 2 Lecture 3

Semester 1, 2026

Acknowledgements

Some of the content in these slides are inspired by the following materials:

- The Word RAM Model by Yufei Tao:

<http://www.cse.cuhk.edu.hk/~taoyf/course/comp3506/lec/ram.pdf>

- The Asymptotic Notation by Yufei Tao:

<http://www.cse.cuhk.edu.hk/~taoyf/course/comp3506/lec/asymptotic.pdf>

Computational Models

About Theoretical Computer Science

Theoretical computer science (TCS) is arguably a branch of **mathematics**.

In Theoretical Computer Science

Everything, e.g., concept, term, claim, symbol, notation and etc., is **abstract** and **mathematically rigorous**.

Algorithm designs are **independent** to **any** specific programming language.

These all hold in this subject.

What Distinguishes Theoretical Computer Science?

Unlike maths and physics, which are to **discover laws** to explain how the world operates, theoretical computer science:

- **defines the laws** of the world (i.e., how a machine operates) in the first place, and
- studies to design “**efficient**” solutions (i.e., the algorithms) to the given problems by **manipulating the laws**.

To start the study of theoretical computer science, we need to define a **computational model** first, which:

- models the machine and specifies what operations it can perform;
- defines how to measure the efficiency of a solution to a problem.

There are **various** computational models. Under different computational models, the philosophy in designing algorithms can be vastly different.

The Word Random Access Machine (RAM) Model

In the Word RAM model, a machine has a **memory** and a **CPU**.

Memory

The memory is a sequence of **cells**:

- each cell can store a **word** in w bits;
- each cell is identified by an **unique address** (encoded by a word of w bits), where the addresses start from 0 for the first cell, 1 for the second, ..., and so on.

CPU

The CPU has a **fixed number** of **registers**, each of which can store a **word** of w bits (i.e., same as a memory cell).

Atomic Operations of the CPU

There are **four** atomic operations that the CPU can perform:

- **Register (Re-)Initialization:** Set the content of a register to a **fixed** value, or to the content of another register.
- **Arithmetic Operations:** Take two integers a and b stored in two registers, calculate the following and then store the result in a register:
 - $a + b$, $a - b$, $a \cdot b$, and **integer division** a/b .
 - For example, $10/2 = 5$, $3/2 = 1$ and $4/6 = 0$.
 - bit operations, like $\wedge$, $\vee$, $\oplus$, $\neg$ and taking the b^{th} -bit of the binary representation of a .

Atomic Operations of the CPU

- **Comparison or Branching:** Compare two integers a and b stored in two registers and decide which of the following is **true**:
 - $a < b$, $a = b$, and $a > b$.
- **Memory Access:** Take the memory address p stored in a register, then perform one of the following:
 - **Read** the content stored in the memory cell at address p into a specified register;
 - **Write** the content stored in a specified register to the memory cell at address p .

Algorithm, Running time Cost, and Space Consumption

Every **algorithm** (under the Word RAM model) is a **sequence** of the above four atomic operations.

The **running time cost** of an algorithm is measured by the **number of operations in the sequence**.

The **space consumption** of an algorithm is measured by the **footprint of memory cells it has ever touched**.

However, the **actual running time** of an algorithm can be affected by various factors.

Question: Can you think of any factor that might affect the actual running time of an algorithm?

Algorithm, Running time Cost, and Space Consumption

Therefore, the **precise number** of atomic operations (rsp., memory cells used) in an algorithm is *less important*.

Instead, people are more interested in **how fast the number of operations (rsp., memory cells) grows** as *a function of the problem size*.

Such a function is called the **running time (rsp., space) complexity** of the algorithm.

Worst Case, Expectation, Amortization

In general, we would be interested to analyse an algorithm in terms of the following different cases:

In the Worst Case: The complexity of an algorithm that works on the *worst-case* input.

In Expectation: The *expected complexity* of an algorithm. The randomness usually comes from either:

- the input distribution, or
- the design of the algorithm.

In Amortization: The *worst-case overall complexity* of an algorithm running on a *sufficiently long sequence of operations*. Only the *amortized complexity* of each operation over the sequence would be of interest, rather than the worst-case individual complexity of each operation.

Asymptotic Notation

Consider two functions, $f(n)$ and $g(n)$, of a variable n .

Big- O Notation

We say $f(n)$ **grows asymptotically no faster than** $g(n)$, if there exist two *positive constants* c_1 and c_2 such that:

$$f(n) \leq c_1 \cdot g(n) \text{ holds for all } n \geq c_2.$$

Big- O notation $O(g(n))$ is defined as a family of functions:

$$O(g(n)) = \{h(n) \mid h(n) \text{ grows asymptotically no faster than } g(n)\}.$$

Therefore, $f(n) \in O(g(n))$.

People may write $f(n) = O(g(n))$ with an abuse of notation.

True or False

$$1000000 \in O(1)$$

$$6n + 7 \in O(n)$$

$$6n + 7 \in O(n^2)$$

$$(\log_2 n)^{999999999} \in O(n^{0.00000000000001})$$

$$\log n \in O(n^\epsilon) \text{ for all constant } \epsilon > 0$$

$$n^{999999999999} \in O(2^n)$$

In general, we have:

$$\begin{aligned} O(1) \subset O(\log n) \subset O(\log^c n) \subset O(n^\epsilon) \\ \subset O(n^c) \subset O(n^{\log n}) \subset O((1+\epsilon)^n) \subset O(n^n), \end{aligned}$$

where ϵ and c are arbitrary constants satisfying $0 < \epsilon < 1 \leq c$.

Big- Ω Notation

We say $f(n)$ **grows asymptotically no slower than** $g(n)$, if there exist two *positive constants* c_1 and c_2 such that:

$$f(n) \geq c_1 \cdot g(n) \text{ holds for all } n \geq c_2.$$

Big- Ω notation $\Omega(g(n))$ is defined as *a family of functions*:

$$\Omega(g(n)) = \{h(n) \mid h(n) \text{ grows asymptotically no slower than } g(n)\}.$$

Therefore, $f(n) \in \Omega(g(n))$, and people may write $f(n) = \Omega(g(n))$.

Examples

$$1000000 \in \Omega(1)$$

$$n^2 + 3 \in \Omega(n)$$

$$n^{0.000000000000001} \in \Omega((\log_2 n)^{999999999})$$

$$2^n \in \Omega(n^{99999999999})$$

Big- Θ Notation

Big- Θ notation $\Theta(g(n))$ is defined as:

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n)),$$

which is the family of all functions of n that **grow asymptotically as fast as $g(n)$** .

In other words, if $f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$ both hold, then $f(n) \in \Theta(g(n))$.

Again, people may write $f(n) = \Theta(g(n))$.